

SOFTWARE ENGINEERING LAB 04

NAME : VAGEESH

SEC : K

SRN : PES2IG24CS663

DATE : 08/09/2026

I'll review every gameplay file together, then tighten the timing and lifecycle rules: add a real firing cooldown, ensure each laser is consumed after a hit, clean child fragments through the same off-screen filter, and make shield/multiplier timers deterministic at frame boundaries.

Read , line 1 to the end

Completed with input: {

```
"path": "C:\\Users\\vagee\\OneDrive\\Desktop\\5th sem\\SE\\vibecoding\\38-meteor-dodge\\game\\game_engine.py",
"view_range": [
  1,
  -1
]
```

Read , line 1 to the end

Completed with input: {

```
"path": "C:\\Users\\vagee\\OneDrive\\Desktop\\5th sem\\SE\\vibecoding\\38-meteor-dodge\\game\\laser.py",
"view_range": [
  1,
  -1
]
```

Read , line 1 to the end

Completed with input: {

```
"path": "C:\\Users\\vagee\\OneDrive\\Desktop\\5th sem\\SE\\vibecoding\\38-meteor-dodge\\game\\meteor.py",
"view_range": [
  1,
  -1
]
```

Read , line 1 to the end

Completed with input: {

```
"path": "C:\\Users\\vagee\\OneDrive\\Desktop\\5th sem\\SE\\vibecoding\\38-meteor-dodge\\game\\energy_orb.py",
"view_range": [
```

```
1,  
-1  
]  
}
```

Read , line 1 to the end

Completed with input: {

```
"path": "C:\\Users\\vagee\\OneDrive\\Desktop\\5th sem\\SE\\vibecoding\\38-meteor-  
dodge\\game\\ship.py",  
"view_range": [  
1,  
-1  
]  
}
```

The review found three concrete issues: unlimited firing on repeated Space presses, one laser potentially destroying multiple meteors in one frame, and shield duration losing a frame immediately on pickup. I'll fix those while preserving child cleanup and the existing multiplier semantics.

Edit

Read , lines 1 to 190

Completed with input: {

```
"path": "C:\\Users\\vagee\\OneDrive\\Desktop\\5th sem\\SE\\vibecoding\\38-meteor-  
dodge\\game\\game_engine.py",  
"view_range": [  
1,  
190  
]  
}
```

pylanceFileSyntaxErrors

Completed with input: {"serverName":"pylance mcp server","toolName":"pylance-mcp-server-pylanceFileSyntaxErrors"}

Ran terminal command: git --no-pager diff --check

Edit

The edge-case fixes are in place: firing is rate-limited, a laser is consumed after its first meteor hit, shield duration now gets the full configured window, and children continue through the normal cleanup filter. I'm doing a final syntax check across all gameplay modules before returning the complete source.

pylanceFileSyntaxErrors

Completed with input: {"serverName":"pylance mcp server","toolName":"pylance-mcp-server-pylanceFileSyntaxErrors"}

pylanceFileSyntaxErrors

Completed with input: {"serverName":"pylance mcp server","toolName":"pylance-mcp-server-pylanceFileSyntaxErrors"}

pylanceFileSyntaxErrors

Completed with input: {"serverName":"pylance mcp server","toolName":"pylance-mcp-server-pylanceFileSyntaxErrors"}

pylanceFileSyntaxErrors

Completed with input: {"serverName":"pylance mcp server","toolName":"pylance-mcp-server-pylanceFileSyntaxErrors"}

pylanceFileSyntaxErrors

Completed with input: {"serverName":"pylance mcp server","toolName":"pylance-mcp-server-pylanceFileSyntaxErrors"}

Ran terminal command: git --no-pager diff --check; git --no-pager status --short

Reviewed and fixed the gameplay edge cases:

- Added an 8-frame laser cooldown.
- A laser is consumed after its first meteor hit, preventing one laser from destroying multiple meteors.
- Large meteor children use the existing off-screen cleanup.
- Shield duration now receives the full 10-second duration.
- Shield still absorbs exactly one collision.
- Fatal collisions reset the multiplier to `x1`.
- Multiplier remains capped at `x5`.

All gameplay files passed syntax validation.

```
### `game/game_engine.py`
```

```
```python
import pygame
import random
```

```
from game.ship import Ship
from game.meteor import Meteor
from game.laser import Laser
from game.energy_orb import EnergyOrb
```

```
WIDTH, HEIGHT = 700, 520
```

```
FPS = 60
BG = (8, 5, 20)
SHIELD_DURATION = FPS * 10
MULTIPLIER_INTERVAL = FPS * 10
MAX_MULTIPLIER = 5
LASER_COOLDOWN = 8
```

```
class GameEngine:
 def __init__(self):
 pygame.init()
 self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
 pygame.display.set_caption("Meteor Dodge")
 self.clock = pygame.time.Clock()
 self.font = pygame.font.SysFont("monospace", 26, bold=True)
 self.big_font = pygame.font.SysFont("monospace", 46, bold=True)

 self.stars = [
 (
 random.randint(0, WIDTH),
 random.randint(0, HEIGHT),
 random.randint(1, 3),
)
 for _ in range(80)
]

 self.reset()

 def reset(self):
 self.ship = Ship(WIDTH // 2, HEIGHT - 80)
 self.meteors = []
 self.lasers = []
 self.energy_orb = None
 self.powerup_timer = random.randint(300, 600)
 self.shield_timer = 0
 self.timer = 0
 self.survival_frames = 0
 self.multiplier_timer = 0
 self.multiplier = 1
 self.laser_cooldown = 0
 self.spawn_interval = 60
 self.score = 0
 self.game_over = False
 self.started = False

 def handle_events(self):
 for event in pygame.event.get():
```

```

if event.type == pygame.QUIT:
 return False

if event.type == pygame.KEYDOWN:
 if event.key == pygame.K_SPACE:
 if self.game_over:
 self.reset()
 elif not self.started:
 self.started = True
 elif self.laser_cooldown == 0:
 self.lasers.append(Laser(*self.ship.rect.midtop))
 self.laser_cooldown = LASER_COOLDOWN

return True

def update(self):
 if self.game_over or not self.started:
 return

 self.survival_frames += 1
 self.multiplier_timer += 1

 if (
 self.multiplier_timer >= MULTIPLIER_INTERVAL
 and self.multiplier < MAX_MULTIPLIER
):
 self.multiplier += 1
 self.multiplier_timer = 0

 keys = pygame.key.get_pressed()
 self.ship.move(keys, WIDTH, HEIGHT)

 self.timer += 1

 if self.timer >= self.spawn_interval:
 self.meteors.append(Meteor(WIDTH))
 self.timer = 0
 self.spawn_interval = max(20, self.spawn_interval - 0.3)

 if self.energy_orb is None:
 self.powerup_timer -= 1

 if self.powerup_timer <= 0:
 self.energy_orb = EnergyOrb(WIDTH)
 self.powerup_timer = random.randint(600, 900)
 else:
 self.energy_orb.update(WIDTH)

```

```

 if self.energy_orb.collides(self.ship.rect):
 self.shield_timer = SHIELD_DURATION
 self.energy_orb = None
 elif self.energy_orb.off_screen(HEIGHT):
 self.energy_orb = None

 for laser in self.lasers:
 laser.update()

 safe_meteors = []

 for meteor in self.meteors:
 meteor.update()

 if meteor.collides(self.ship.rect):
 if self.shield_timer > 0:
 self.shield_timer = 0
 continue

 self.game_over = True
 self.multiplier = 1
 self.multiplier_timer = 0

 safe_meteors.append(meteor)

 self.meteors = safe_meteors

 remaining_meteors = []
 used_lasers = set()

 for meteor in self.meteors:
 hit_laser = next(
 (
 index
 for index, laser in enumerate(self.lasers)
 if index not in used_lasers
 and laser.rect.collidepoint(
 int(meteor.x),
 int(meteor.y),
)
),
 None,
)

 if hit_laser is None:
 remaining_meteors.append(meteor)

```

```

 else:
 used_lasers.add(hit_laser)
 remaining_meteors.extend(meteor.split())

self.meteors = remaining_meteors

self.lasers = [
 laser
 for index, laser in enumerate(self.lasers)
 if index not in used_lasers and not laser.off_screen()
]

self.meteors = [
 meteor
 for meteor in self.meteors
 if not meteor.off_screen(HEIGHT)
]

self.score += self.multiplier

if self.laser_cooldown > 0:
 self.laser_cooldown -= 1

if self.shield_timer > 0:
 self.shield_timer -= 1

def draw(self):
 self.screen.fill(BG)

 for sx, sy, sr in self.stars:
 pygame.draw.circle(
 self.screen,
 (200, 200, 220),
 (sx, sy),
 sr,
)

 for meteor in self.meteors:
 meteor.draw(self.screen)

 if self.energy_orb is not None:
 self.energy_orb.draw(self.screen)

 for laser in self.lasers:
 laser.draw(self.screen)

 self.ship.draw(self.screen)

```

```

if self.shield_timer > 0:
 pygame.draw.circle(
 self.screen,
 (90, 220, 255),
 self.ship.rect.center,
 30,
 3,
)

score_text = self.font.render(
 f"Time: {self.survival_frames // 60}s",
 True,
 (200, 200, 240),
)
self.screen.blit(score_text, (10, 10))

multiplier_text = self.font.render(
 f"x{self.multiplier}",
 True,
 (255, 220, 100),
)
self.screen.blit(
 multiplier_text,
 (WIDTH - multiplier_text.get_width() - 10, 10),
)

if not self.started:
 message = self.font.render(
 "Press SPACE to launch",
 True,
 (180, 180, 240),
)
 self.screen.blit(
 message,
 (
 WIDTH // 2 - message.get_width() // 2,
 HEIGHT // 2,
),
)

if self.game_over:
 overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
 overlay.fill((0, 0, 0, 150))
 self.screen.blit(overlay, (0, 0))

 message = self.big_font.render(

```

```

 "DESTROYED!",
 True,
 (220, 80, 60),
)
 submessage = self.font.render(
 f"Survived {self.survival_frames // 60}s | SPACE to Restart",
 True,
 (200, 200, 200),
)

 self.screen.blit(
 message,
 (
 WIDTH // 2 - message.get_width() // 2,
 HEIGHT // 2 - 40,
),
)
 self.screen.blit(
 submessage,
 (
 WIDTH // 2 - submessage.get_width() // 2,
 HEIGHT // 2 + 20,
),
)

pygame.display.flip()

def run(self):
 running = True

 while running:
 running = self.handle_events()
 self.update()
 self.draw()
 self.clock.tick(FPS)

 pygame.quit()
...

`game/laser.py`

```python
import pygame

SPEED = 8

```

```

class Laser:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x - 2, y - 18, 4, 18)

    def update(self):
        self.rect.y -= SPEED

    def off_screen(self):
        return self.rect.bottom < 0

    def draw(self, screen):
        pygame.draw.rect(screen, (100, 220, 255), self.rect)
        pygame.draw.rect(
            screen,
            (220, 250, 255),
            self.rect.inflate(-2, 0),
        )
    ...

```

```

### `game/meteor.py`

```

```

```python
import pygame
import random
import math

```

```

LARGE_RADIUS = 20

```

```

class Meteor:
 def __init__(
 self,
 width=None,
 x=None,
 y=None,
 radius=None,
 vx=None,
 vy=None,
 color=None,
):
 self.x = random.randint(0, width) if x is None else x
 self.y = -30 if y is None else y
 self.radius = random.randint(12, 28) if radius is None else radius

 if vx is None or vy is None:

```

```

 angle = random.uniform(70, 110)
 speed = random.uniform(2, 5)
 self.vx = math.cos(math.radians(angle)) * speed
 self.vy = math.sin(math.radians(angle)) * speed
else:
 self.vx = vx
 self.vy = vy

if color is None:
 color = (
 random.randint(160, 220),
 random.randint(80, 120),
 random.randint(40, 80),
)

self.color = color
self.rot = 0
self.rot_speed = random.uniform(-3, 3)

def split(self):
 if self.radius < LARGE_RADIUS:
 return []

 child_radius = max(8, self.radius // 2)
 child_count = random.randint(2, 3)
 direction = math.atan2(self.vy, self.vx)
 children = []

 for index in range(child_count):
 spread = math.radians(random.uniform(25, 55))

 if index % 2:
 spread *= -1

 angle = direction + spread
 speed = random.uniform(2.5, 4.5)

 children.append(
 Meteor(
 x=self.x,
 y=self.y,
 radius=child_radius,
 vx=math.cos(angle) * speed,
 vy=math.sin(angle) * speed,
 color=self.color,
)
)

```

```
return children
```

```
def update(self):
 self.x += self.vx
 self.y += self.vy
 self.rot = (self.rot + self.rot_speed) % 360
```

```
def off_screen(self, height):
 return self.y > height + 60
```

```
def collides(self, rect):
 cx, cy = rect.centerx, rect.centery
 dx, dy = self.x - cx, self.y - cy
 return (dx**2 + dy**2) ** 0.5 < self.radius + 16
```

```
def draw(self, screen):
 pts = []

 for i in range(7):
 angle = math.radians(self.rot + i * (360 / 7))
 r = self.radius * (0.8 + 0.2 * (i % 2))
 pts.append(
 (
 int(self.x + r * math.cos(angle)),
 int(self.y + r * math.sin(angle)),
)
)
```

```
pygame.draw.polygon(screen, self.color, pts)
```

```
inner = [
 (
 int(
 self.x
 + (self.radius * 0.5)
 * math.cos(
 math.radians(self.rot + i * (360 / 7))
)
),
 int(
 self.y
 + (self.radius * 0.5)
 * math.sin(
 math.radians(self.rot + i * (360 / 7))
)
),
)
```

```

)
 for i in range(7)
]

 pygame.draw.polygon(
 screen,
 tuple(max(0, c - 40) for c in self.color),
 inner,
)
...

`game/energy_orb.py`

```python
import math
import random

import pygame

class EnergyOrb:
    def __init__(self, width):
        self.radius = 10
        self.x = random.randint(self.radius, width - self.radius)
        self.y = -self.radius
        self.vx = random.uniform(-0.6, 0.6)
        self.vy = random.uniform(0.4, 0.8)
        self.phase = random.uniform(0, math.tau)

    def update(self, width):
        self.x += self.vx
        self.y += self.vy
        self.phase += 0.08

        if self.x <= self.radius or self.x >= width - self.radius:
            self.vx *= -1

    def off_screen(self, height):
        return self.y > height + self.radius

    def collides(self, rect):
        closest_x = max(rect.left, min(self.x, rect.right))
        closest_y = max(rect.top, min(self.y, rect.bottom))
        dx = self.x - closest_x
        dy = self.y - closest_y
        return dx * dx + dy * dy <= self.radius**2

```

```

def draw(self, screen):
    pulse = 2 + int((math.sin(self.phase) + 1) * 2)
    center = (int(self.x), int(self.y))

    pygame.draw.circle(
        screen,
        (80, 220, 255),
        center,
        self.radius + pulse,
        2,
    )
    pygame.draw.circle(
        screen,
        (120, 240, 255),
        center,
        self.radius,
    )
    pygame.draw.circle(
        screen,
        (235, 255, 255),
        center,
        self.radius // 2,
    )
...

```

```

### `game/ship.py`

```

```

``python
import pygame

```

```

SPEED = 5

```

```

class Ship:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x - 20, y - 20, 40, 40)
        self.color = (80, 160, 240)
        self.trail = []

    def move(self, keys, width, height):
        dx = dy = 0

        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            dx = -SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            dx = SPEED

```

```

if keys[pygame.K_UP] or keys[pygame.K_w]:
    dy = -SPEED
if keys[pygame.K_DOWN] or keys[pygame.K_s]:
    dy = SPEED

self.rect.x = max(
    0,
    min(width - self.rect.width, self.rect.x + dx),
)
self.rect.y = max(
    0,
    min(height - self.rect.height, self.rect.y + dy),
)

self.trail.append(tuple(self.rect.center))

if len(self.trail) > 10:
    self.trail.pop(0)

def draw(self, screen):
    for i, pos in enumerate(self.trail):
        alpha = 20 + i * 20
        r = 3 + i // 2
        surface = pygame.Surface((r * 2, r * 2), pygame.SRCALPHA)
        pygame.draw.circle(
            surface,
            (80, 160, 240, alpha),
            (r, r),
            r,
        )
        screen.blit(surface, (pos[0] - r, pos[1] - r))

cx, cy = self.rect.center
points = [
    (cx, cy - 18),
    (cx - 14, cy + 14),
    (cx, cy + 6),
    (cx + 14, cy + 14),
]

pygame.draw.polygon(screen, self.color, points)
pygame.draw.circle(
    screen,
    (255, 180, 60),
    (cx, cy + 12),
    5,
)

```

